

APOLLO INSTITUTE OF COMPUTER APPLICATION

BCA SEM2- ASSIGNMENT

SUBJECT: DATA STRUCTURE

UNIT 3: NONLINEAR DATA STRUCTURE

Short Questions

1. Define Tree in Data Structure

A **Tree** is a **non-linear hierarchical data structure** consisting of nodes connected by edges. It has one **root node** and multiple **child nodes**, forming a parent–child relationship.

2. Define the following terms of Tree

Root Node

The **topmost node** of a tree is called the **Root Node**. It has **no parent**.

Leaf Node

A node that **does not have any child nodes** is called a **Leaf Node**.

Degree of Node

The **number of children** of a node is called its **degree**.

Level of Node

The **position of a node in the tree hierarchy** is called its level. Root node is usually at **Level 0 or Level 1**.

3. What is a Binary Tree?

A **Binary Tree** is a tree data structure in which **each node has at most two children**, called **left child** and **right child**.

4. Write different types of Binary Trees

Types of Binary Trees:

1. **Full Binary Tree**
2. **Complete Binary Tree**
3. **Perfect Binary Tree**

4. **Balanced Binary Tree**
 5. **Binary Search Tree (BST)**
 6. **Threaded Binary Tree**
-

5. What is Binary Tree Traversal?

Binary Tree Traversal is the **process of visiting every node of a binary tree exactly once in a specific order**.

6. Write the three types of Binary Tree Traversal

1. **Inorder Traversal**
Left → Root → Right
 2. **Preorder Traversal**
Root → Left → Right
 3. **Postorder Traversal**
Left → Right → Root
-

7. What is a Threaded Binary Tree?

A **Threaded Binary Tree** is a binary tree in which **null pointers are replaced with links (threads) to the inorder predecessor or successor**, allowing faster traversal.

8. Define Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree in which:

- Left subtree contains **values smaller than root**
 - Right subtree contains **values greater than root**
-

9. Write two applications of Trees

1. **File System organization**
2. **Database indexing**
3. **Expression evaluation**
4. **Decision trees**

(Write any two)

10. Define Graph in Data Structure

A **Graph** is a non-linear data structure consisting of **vertices (nodes)** and **edges (connections)** between them.

11. What is a Spanning Tree?

A **Spanning Tree** is a **subgraph of a connected graph** that includes **all vertices with minimum edges and no cycles**.

For a graph with **n vertices**, spanning tree has **n-1 edges**.

12. Define Minimal Spanning Tree (MST)

A **Minimal Spanning Tree (MST)** is a spanning tree in which the **total weight of all edges is minimum**.

Medium Questions

1. Explain Representation of Binary Tree using Array with Example

Binary Tree can be represented using **Array Representation**, where all nodes of the tree are stored in an array.

In this method, nodes are stored **sequentially in array positions** according to their relationship.

Rules for Array Representation

If a node is stored at index **i**:

- **Left Child = $2i$**
- **Right Child = $2i + 1$**
- **Parent Node = $i / 2$**

Example

Binary Tree:



```

      /  \   \
     D    E   F

```

Array Representation

Index 1 2 3 4 5 6

Value A B C D E F

Explanation:

- A is root at **index 1**
- B is **left child of A**
- C is **right child of A**
- D and E are children of **B**
- F is right child of **C**

Advantages

- Simple implementation
- Easy access to nodes

Disadvantages

- Wastes memory if tree is **not complete**

v

2. Explain Adjacency Matrix Representation of Graph

In **Adjacency Matrix Representation**, a graph is represented using a **two-dimensional matrix**.

- Rows represent **vertices**
- Columns represent **vertices**

If there is an edge between two vertices, the value is **1**, otherwise **0**.

Example Graph

Vertices: **A, B, C**

Edges:

- A – B
- B – C

Adjacency Matrix

A B C

A 0 1 0

B 1 0 1

C 0 1 0

Explanation:

- **1 indicates connection**
- **0 indicates no connection**

Advantages

- Easy to implement
- Fast edge lookup

Disadvantages

- Uses **more memory for large graphs**

3. Explain Breadth First Search (BFS)

Breadth First Search (BFS) is a graph traversal algorithm that visits nodes **level by level**.

It uses a **Queue data structure**.

Steps of BFS

1. Start from the **source node**.
2. Visit the node and mark it as visited.
3. Insert all **adjacent nodes into the queue**.
4. Remove node from queue.
5. Repeat until all nodes are visited.

Example

Graph:

A

/ \

B C

/ \

D E

BFS Traversal

A → B → C → D → E

Applications

- Shortest path in unweighted graphs
 - Social networking analysis
 - Web crawling
-

4. Explain Depth First Search (DFS)

Depth First Search (DFS) is a graph traversal method that explores **as deep as possible before backtracking**.

It uses **Stack or Recursion**.

Steps of DFS

1. Start from the **root node**.
 2. Visit the node.
 3. Move to the **next unvisited adjacent node**.
 4. Continue until no node is left.
 5. Backtrack and explore remaining nodes.
-

Example

Graph:

A

/ \

B C

/ \

D E

DFS Traversal

A → B → D → E → C

Applications

- Cycle detection
- Path finding
- Topological sorting

5. Explain Shortest Path Problem in Graph

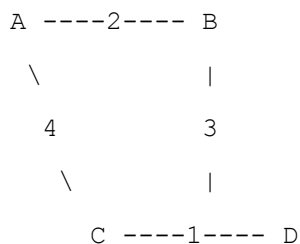
The **Shortest Path Problem** is used to find the **minimum distance between two vertices in a graph**.

Each edge in the graph may have a **weight (cost or distance)**.

The goal is to find the path with **minimum total weight**.

Example

Graph:



To find the shortest path from **A to D**:

Possible paths:

- $A \rightarrow B \rightarrow D = 2 + 3 = 5$
- $A \rightarrow C \rightarrow D = 4 + 1 = 5$

Shortest distance = 5

Common Algorithms

1. Dijkstra's Algorithm
 2. Bellman-Ford Algorithm
 3. Floyd-Warshall Algorithm
-

Applications

- GPS Navigation systems

- Network routing
- Airline route planning
- Transportation systems

Long Questions

1. Explain Binary Tree Representation with Suitable Example

A **Binary Tree** is a tree data structure in which each node can have **at most two children**, called **Left Child** and **Right Child**.

Binary Trees can be represented in two ways:

1. **Array Representation**
2. **Linked List Representation**

1. Array Representation

In array representation, the nodes of a binary tree are stored **sequentially in an array**.

Rules

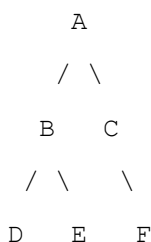
If a node is stored at index **i**:

- Left Child = $2i$
- Right Child = $2i + 1$
- Parent Node = $i / 2$

This representation works best for **complete binary trees**.

Example

Binary Tree:



Array Representation:

Index 1 2 3 4 5 6

Value A B C D E F

Explanation:

- A is stored at position 1
- B and C are children of A
- D and E are children of B
- F is right child of C

Advantages

- Simple implementation
- Direct access to nodes

Disadvantages

- Wastes memory for sparse trees.

2. Linked List Representation

In this method, each node contains:

- Data
- Pointer to Left Child
- Pointer to Right Child

Structure of Node

Node

```
{  
    Data  
    Left Pointer  
    Right Pointer  
}
```

Advantages

- Efficient memory usage
- Dynamic structure

2. Explain the following Binary Tree Traversal methods with example: Inorder Traversal Preorder Traversal Postorder Traversal

Binary Tree Traversal is the process of **visiting each node of a binary tree exactly once in a specific order**. Traversal helps in **processing, searching, or printing the elements** of a binary tree.

The three important traversal methods are:

1. **Inorder Traversal (LNR)**
2. **Preorder Traversal (NLR)**
3. **Postorder Traversal (LRN)**

Where:

- **L = Left Subtree**
- **N = Node (Root)**
- **R = Right Subtree**

Consider the following Binary Tree:

```
A
 / \
B   C
 / \ / \
D E F G
```

1. Inorder Traversal (LNR)

In **Inorder Traversal**, the nodes are visited in the following order:

Left Subtree → Root → Right Subtree

Steps

1. Traverse the left subtree.
2. Visit the root node.
3. Traverse the right subtree.

Example

For the above tree:

Left → Root → Right

Traversal result:

D → B → E → A → F → C → G

Algorithm

Inorder(root)

```
{  
    if(root ≠ NULL)  
    {  
        Inorder(root->left)  
        Print(root->data)  
        Inorder(root->right)  
    }  
}
```

Application

- Used in **Binary Search Tree** to get elements in **sorted order**.

2. Preorder Traversal (NLR)

In **Preorder Traversal**, nodes are visited in this order:

Root → Left Subtree → Right Subtree

Steps

1. Visit the root node.
2. Traverse the left subtree.
3. Traverse the right subtree.

Example

Traversal result:

A → B → D → E → C → F → G

Algorithm

Preorder(root)

```
{
```

```

if(root ≠ NULL)
{
    Print(root->data)
    Preorder(root->left)
    Preorder(root->right)
}
}

```

Application

- Used to **create a copy of a tree**.
- Used in **expression trees**.

3. Postorder Traversal (LRN)

In **Postorder Traversal**, nodes are visited in the order:

Left Subtree → Right Subtree → Root

Steps

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the root node.

Example

Traversal result:

$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$

Algorithm

```

Postorder(root)
{
    if(root ≠ NULL)
    {
        Postorder(root->left)
        Postorder(root->right)
        Print(root->data)
    }
}

```

}

Application

- Used for **deleting a tree**.
- Used for **expression evaluation**.

Comparison of Traversal Methods

Traversal Type	Visiting Order	Output (Example Tree)
Inorder	Left → Root → Right	D B E A F C G
Preorder	Root → Left → Right	A B D E C F G
Postorder	Left → Right → Root	D E B F G C A

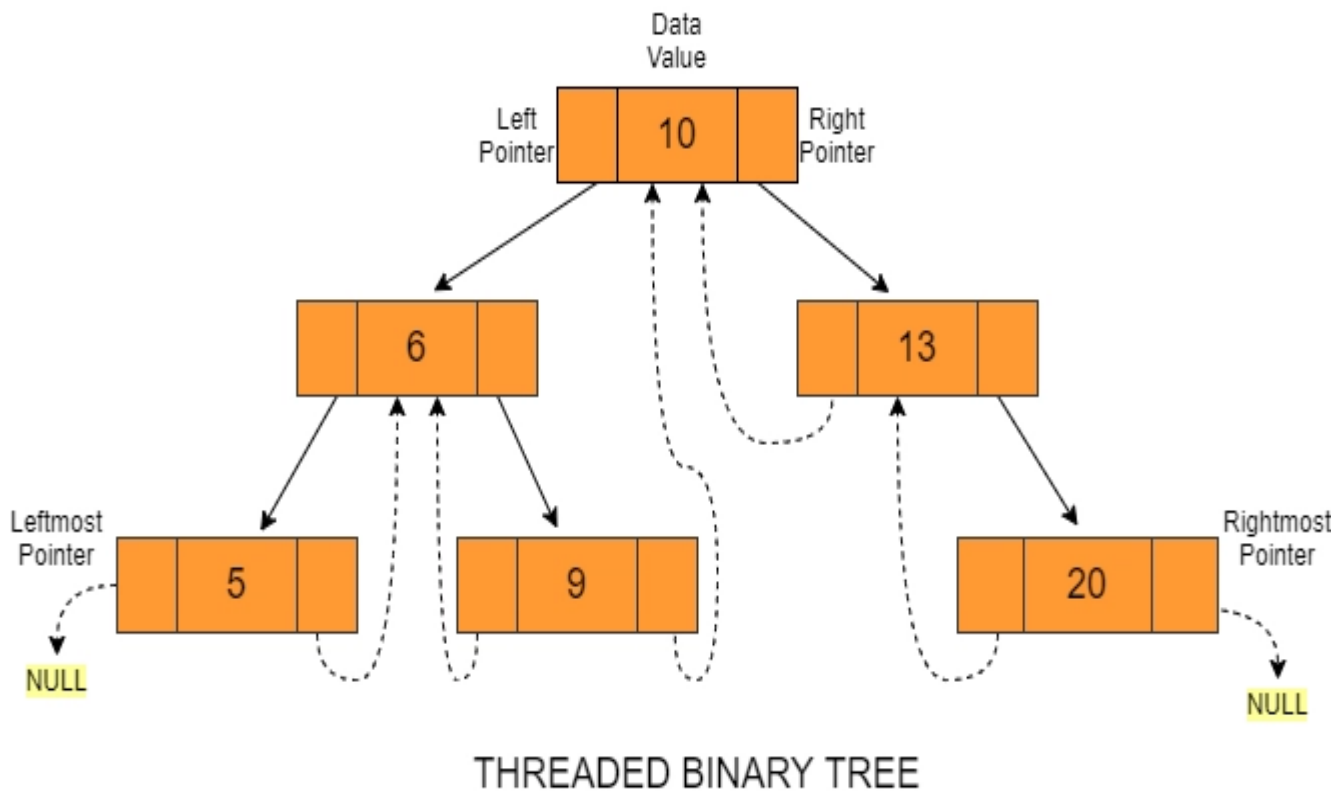
3. Explain Threaded Binary Tree with diagram and advantages.

A **Threaded Binary Tree** is a special type of binary tree in which **NULL pointers are replaced with special links called threads**.

In a normal binary tree, many left or right child pointers are **NULL**. These NULL pointers do not store any useful information. In a **Threaded Binary Tree**, these NULL pointers are used to store the **address of the inorder predecessor or inorder successor**.

This makes **tree traversal easier and faster without using recursion or stack**.

Diagram of Threaded Binary Tree



In the diagram:

- **Solid lines** represent normal parent-child relationships.
- **Dashed lines (threads)** represent links to the **inorder successor or predecessor**.

Types of Threaded Binary Trees

1. Single Threaded Binary Tree

In this tree, only **one type of thread** is used.

- **Left Threaded Tree** – left NULL pointers point to inorder predecessor.
- **Right Threaded Tree** – right NULL pointers point to inorder successor.

2. Double Threaded Binary Tree

In this tree, **both left and right NULL pointers are replaced with threads**.

- Left pointer → inorder predecessor
- Right pointer → inorder successor

This allows traversal **in both directions**.

Advantages of Threaded Binary Tree

1. **Efficient Traversal**

Tree traversal can be done without recursion or stack.

2. **Better Memory Utilization**

NULL pointers are used as threads instead of remaining unused.

3. **Faster Inorder Traversal**

Threads directly link nodes in traversal order.

4. **Simplifies Tree Traversal Algorithms**

Traversal logic becomes simpler.

5. **Reduces Overhead of Stack Operations**

No need for additional data structures during traversal.

4. Explain Binary Search Tree (BST) with operations: Insertion Searching

1. A **Binary Search Tree (BST)** is a special type of binary tree in which the nodes are arranged according to the following rules:
2. All elements in the **left subtree** are **less than the root node**.
3. All elements in the **right subtree** are **greater than the root node**.
4. Both the **left and right subtrees** are also **Binary Search Trees**.

This property makes **searching, insertion, and deletion operations efficient**.

Example of Binary Search Tree

Operations on Binary Search Tree

The two main operations are:

1. **Insertion**
 2. **Searching**
-

1. Insertion in Binary Search Tree

Insertion means **adding a new node to the BST at the correct position** while maintaining the BST property.

Steps for Insertion

1. Start from the **root node**.
2. Compare the new value with the root.
3. If the value is **smaller**, move to the **left child**.
4. If the value is **greater**, move to the **right child**.
5. Repeat the process until a **NULL position is found**.
6. Insert the new node there.

Example

Insert **25** into the BST.

```
    50
   /  \
  30   70
 /  \
20  40
```

Since $25 < 50 \rightarrow$ move left

Since $25 < 30 \rightarrow$ move left

Since $25 > 20 \rightarrow$ insert as **right child of 20**

Final Tree:

```
    50
   /  \
  30   70
 /  \
20  40
 \
  25
```

2. Searching in Binary Search Tree

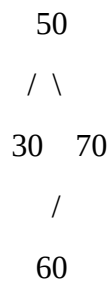
Searching means **finding whether a particular element exists in the BST.**

Steps for Searching

1. Start from the **root node**.
2. Compare the search value with the root.
3. If equal → **element found**.
4. If smaller → search in **left subtree**.
5. If greater → search in **right subtree**.
6. Continue until element is found or **NULL node is reached**.

Example

Search **60** in the BST.



Steps:

- $60 > 50 \rightarrow$ move right
- $60 < 70 \rightarrow$ move left
- 60 found

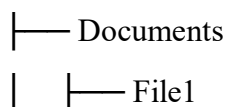
5. Write Applications of Trees in Data Structure.

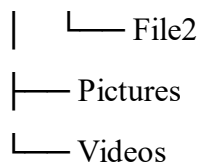
1. File System Organization

Trees are used to represent the **directory structure of operating systems.**

Example:

Root





Each folder is a **node**, and files are stored as **child nodes**.

Advantages:

- Easy navigation
- Organized storage
- Fast searching of files

2. Database Indexing

Trees such as **B-Trees and B+ Trees** are used in databases for **indexing records**.

These trees allow:

- Fast searching
- Efficient insertion and deletion
- Balanced structure

Databases like **MySQL and Oracle** use tree structures for indexing.

3. Expression Trees

Trees are used to represent **mathematical expressions**.

Example:

Expression:

$(A + B) * C$

Tree representation:

*

/ \

+ C

/ \

A B

Expression trees are used in:

- **Compilers**

- **Expression evaluation**
 - **Syntax analysis**
-

4. Searching (Binary Search Tree)

Binary Search Trees (BST) are used for **efficient searching and sorting**.

Example operations:

- Searching
- Insertion
- Deletion

Average time complexity:

$O(\log n)$

BSTs are used in:

- Dictionaries
 - Phonebooks
 - Data indexing
-

5. Network Routing

Trees are used in **computer networks** to represent routing paths.

Example:

- **Spanning Tree Protocol (STP)** used in networks
 - Helps prevent loops
 - Optimizes communication paths
-

6. Artificial Intelligence and Decision Making

Trees are used in **decision-making systems**.

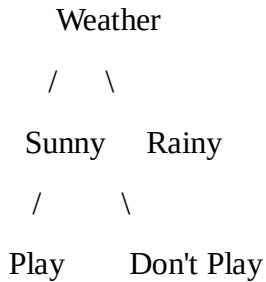
Example: **Decision Trees**

Applications:

- Machine learning
- Data analysis

- Medical diagnosis
- Business decision making

Example structure:



6. Explain Balanced Trees and describe: AVL Tree 2-3 Tree Height Balanced Tree Weight Balanced Tree

A **Balanced Tree** is a type of tree in which the **height of the tree is kept as small as possible** so that operations like **searching, insertion, and deletion can be performed efficiently**.

In a balanced tree, the difference between the heights of the left and right subtrees is **limited**, preventing the tree from becoming skewed.

Balanced trees improve the **time complexity of operations to approximately $O(\log n)$** .

Types of Balanced Trees

The important balanced tree mechanisms are:

1. **AVL Tree**
2. **Height Balanced Tree**
3. **Weight Balanced Tree**

1. AVL Tree

An **AVL Tree** is a self-balancing binary search tree where the **difference between the heights of left and right subtrees of any node is at most 1**.

This difference is called the **Balance Factor**.

Balance Factor

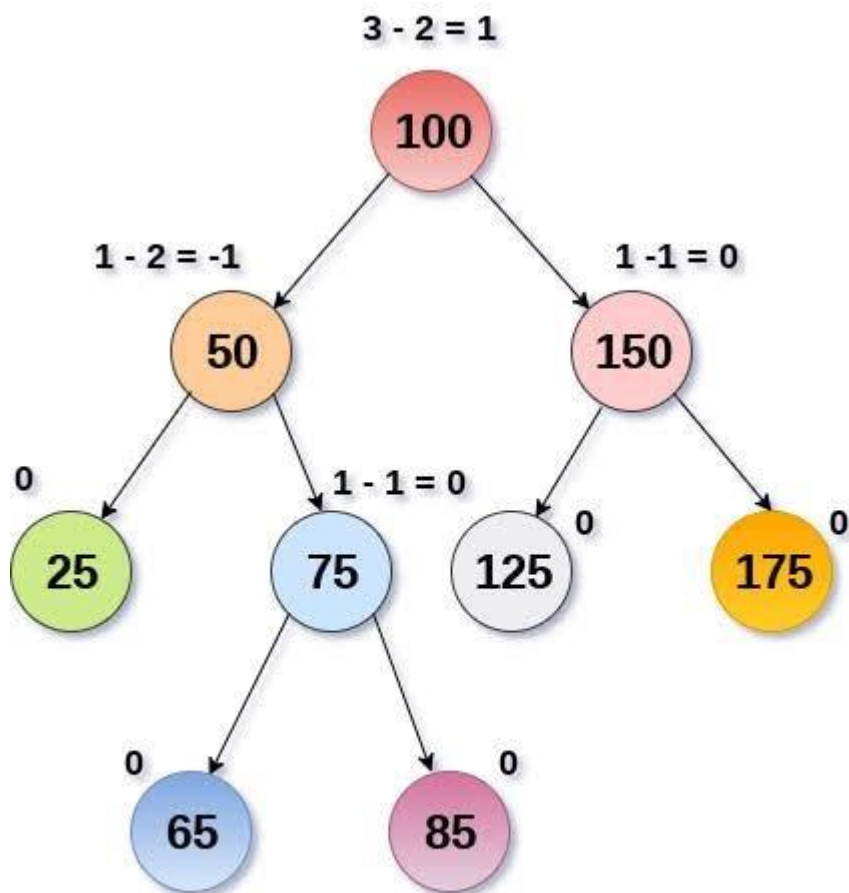
Balance Factor = Height of Left Subtree – Height of Right Subtree

Possible values:

-1, 0, +1

If the balance factor becomes greater than 1 or less than -1, **tree rotations are performed** to balance the tree.

AVL Tree Example



AVL Tree

Rotations used in AVL Tree

1. Left Rotation
2. Right Rotation
3. Left-Right Rotation
4. Right-Left Rotation

Advantages

- Fast searching
 - Automatically balanced
 - Efficient insertion and deletion
-

3. Height Balanced Tree

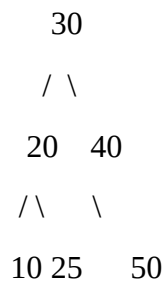
A **Height Balanced Tree** is a tree in which the **height difference between the left and right subtrees of every node is limited**.

Condition:

$$| \text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree}) | \leq 1$$

This ensures that the tree remains balanced and avoids skewed structures.

Example:



Height Calculation

Left Subtree Height = 2

Right Subtree Height = 2

$$|2 - 2| = 0 \leq 1$$

So the tree is **Height Balanced**.

Example:

- **AVL Tree is a type of Height Balanced Tree.**

Advantages

- Improves searching performance

- Prevents tree degeneration

4. Weight Balanced Tree

A **Weight Balanced Tree** maintains balance based on the **number of nodes in the left and right subtrees** instead of height.

Condition:

The number of nodes in both subtrees must remain **approximately equal**.

Example:

Left subtree nodes $\approx$ Right subtree nodes

If the difference becomes large, **restructuring or rotations** are performed.

Example:

```
      50
     /  \
    30   70
   /\   /\
  20 40 60 80
```

In this tree:

Left Subtree Nodes = 3

Right Subtree Nodes = 3

Since both sides have **almost equal number of nodes**, the tree is **weight balanced**.

Advantages

- Better distribution of nodes
- Efficient searching and insertion
- Maintains balanced structure

7. What is a Spanning Tree? Explain with example.

A **Spanning Tree** is a subgraph of a connected graph that includes **all the vertices of the graph** and forms a **tree structure without any cycles**.

In other words, a spanning tree connects all the vertices of the graph using the **minimum possible number of edges**.

If a graph has **n vertices**, then its spanning tree will have exactly:

$n - 1$ edges

Original Graph

A ----- B

| \ |

| \ |

C ----- D

This graph contains **4 vertices** and multiple edges forming **cycles**.

One Possible Spanning Tree

A ----- B

|

C

|

D

In this spanning tree:

- All vertices **A, B, C, D** are connected.
- Number of edges = **3 (n-1)**.
- **No cycles are present.**

Properties of Spanning Tree

1. A spanning tree contains **all vertices of the original graph**.
2. It has **no cycles**.
3. For **n vertices**, it always contains **n - 1 edges**.
4. Removing any edge will **disconnect the tree**.
5. Adding any extra edge will **create a cycle**.

Number of Possible Spanning Trees

A graph can have **more than one spanning tree** depending on which edges are selected.

Different combinations of edges can produce **different spanning trees**.

Applications of Spanning Tree

1. **Network Design** – used to design communication networks with minimum connections.

2. **Electrical Networks** – helps design wiring systems efficiently.
3. **Computer Networks** – used in **Spanning Tree Protocol (STP)** to prevent loops.
4. **Optimization Problems** – used in algorithms related to graph theory.

8. Explain Minimal Spanning Tree (MST) and its importance.

A **Minimal Spanning Tree (MST)** or **Minimum Spanning Tree** is a spanning tree of a **connected weighted graph** in which the **sum of the weights of all edges is minimum**.

In other words, it connects **all vertices of the graph with the minimum possible total edge weight** and **without forming any cycle**.

If a graph has **n vertices**, then the minimum spanning tree will contain:

$n - 1$ edges

Example of Minimum Spanning Tree



Possible **Minimum Spanning Tree**:

A ----- C (2)

C ----- D (1)

D ----- B (3)

Total Weight:

$$2 + 1 + 3 = 6$$

This is the **minimum cost to connect all vertices**.

Algorithms Used to Find MST

Two famous algorithms are used:

1. Prim's Algorithm

- Starts from a **single vertex**.
- Repeatedly adds the **minimum weight edge** that connects a new vertex.

2. Kruskal's Algorithm

- Sorts all edges by **increasing weight**.
- Adds edges one by one **without forming cycles**.

Importance of Minimum Spanning Tree

1. Network Design

Used to design networks such as **telephone, internet, and electrical networks** with minimum cost.

2. Transportation Planning

Helps in designing **roads, railways, and pipelines** with minimum construction cost.

3. Computer Networks

Used in **Spanning Tree Protocol (STP)** to prevent loops in network topology.

4. Circuit Design

Used in designing **efficient electronic circuits**.

5. Optimization Problems

Helps in solving problems that require **minimum connection cost**.